EE5327 VLSI Design Project, Report 1

Submitted by: Joel Kuehne, Email ID: kuehn348@umn.edu

Introduction

Motion estimation is the most computationally intensive task in H.264/H.265 video encoding, accounting for around 80% of the total computational cost for video encoding [1]. As the name implies, motion estimators improve encoding efficiency by identifying movement over time in the source video. The objective of any motion estimation algorithm is to produce an array of optimal motion vectors (MVs) that describe change over time between the current frame and a previous reference frame. Compression standards like Advanced Video Coding (AVC/H.264) and High Efficiency Video Coding (HEVC/H.265) support variable-block-size prediction. In H.264, a variable-block-size motion estimation algorithm produces the optimal displacement vector for each 16x16 macroblock and all of its supported subdivisions (4x4, 4x8, 8x4, 8x8, 8x16, and 16x8).

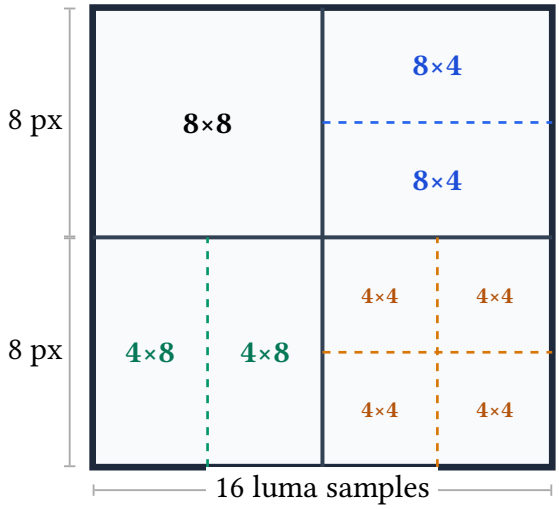


Figure 1: Example of the different subdivisions of a 16 × 16 macroblock in H.264.

Formally, finding the best motion vector can be modeled as finding the minimum of a distance objective function over a finite search window. Sum of Absolute Differences (SAD) is the objective function of choice for almost all motion estimation algorithms. Various architectures and algorithms have been proposed for efficient motion estimation. In hardware, fast-search algorithms such as three-step search (TSS), four-step search (4SS), and diamond search (DS) achieve better computational efficiency by performing a sequential search of the objective function. Advanced fast-search algorithms like Enhanced Predictive Zonal Search (EPZS) and variants of the cross-diamond-hexagonal search achieve high encoding efficiency at a lower compute cost [2]. These advanced fast-search algorithms work well in software, but do not map well to hardware.

Many fast-search motion estimation algorithms assume a unimodal objective function with a global minimum. This assumption is often invalid for real video [3]. Full-search motion estimation abandons this sequential search approach and produces the absolute minimum over the finite search window by computing and comparing all possible SAD results for every subdivision of every macroblock. Because full-search motion estimation is guaranteed to find the minimum SAD over the search space, it achieves higher encoding efficiencies than sequential fast-search methods.

Architecture

Original Design

The key idea behind the original paper "VLSI Architecture of Full-Search Variable-Block-Size Motion Estimation for HEVC Video Encoding" is that SAD results for small block sizes can be merged to form SAD results for larger blocks. Data is streamed into a compute module that computes first-level SADs directly. Data from the first-level SADs is merged recursively in a series of summation modules that build larger SADs from smaller SADs. Various SAD comparators are placed along the datapath to extract the minimum SAD value and associated motion vector.

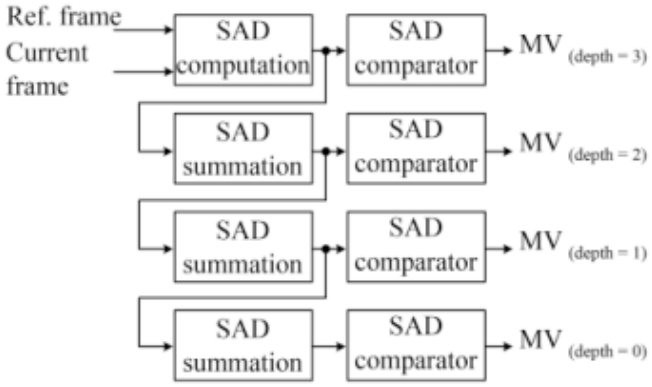


Figure 2: Image of the original datapath used in [3].

The core of the SAD computation block is the processing element (PE) array. Reference frame data is broadcast to an array of 4x4 SAD compute units. Each element in the array reads four pixels from the reference frame and four pixels from the current frame to compute one row of a 4x4 SAD using a 4-input adder tree and an accumulator register.

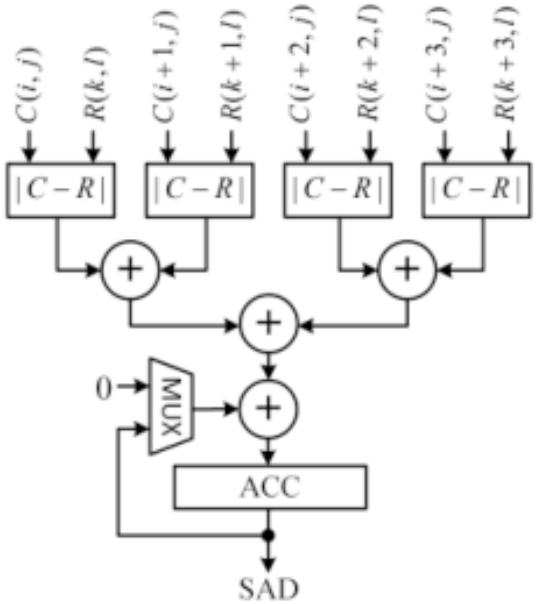


Figure 3: Image of the original PE used in [3].

Each PE computes one 4x4 SAD every four cycles. Each PE array contains several columns equal to the configured search window size and exactly four rows. This allows the PE array to compute every possible SAD value in real time as rows are scanned in from the reference block.

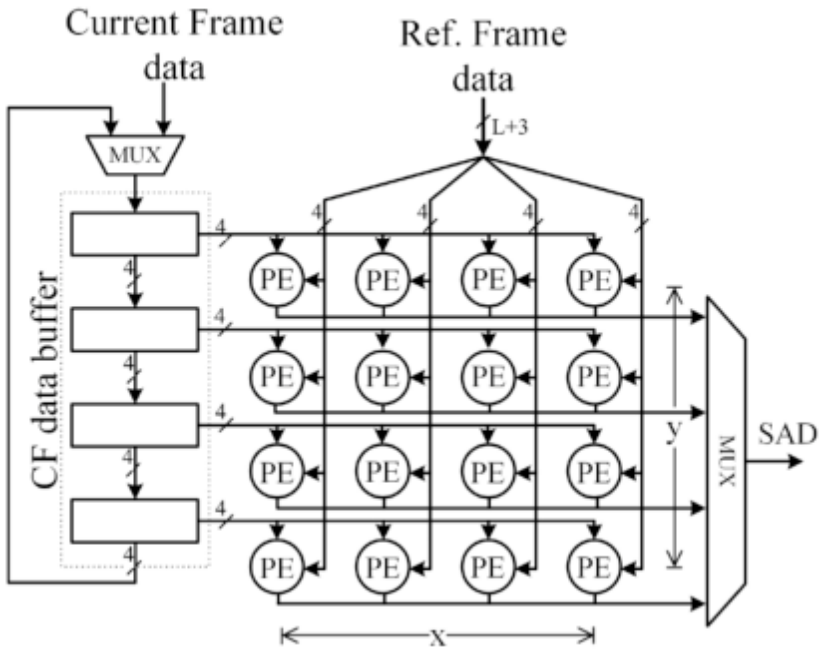


Figure 4: Image of the original PE array used in [3].

The compute module architecture in [3] instantiates four PE arrays, and therefore computes 4x4, 8x4, and 8x8 SADs in one step. These smaller SADs are streamed into SAD summation blocks. This SAD summation block uses three SRAMs to hold old SAD values. Four 8x8 SAD blocks are needed to compute the full 16x16 SAD block.

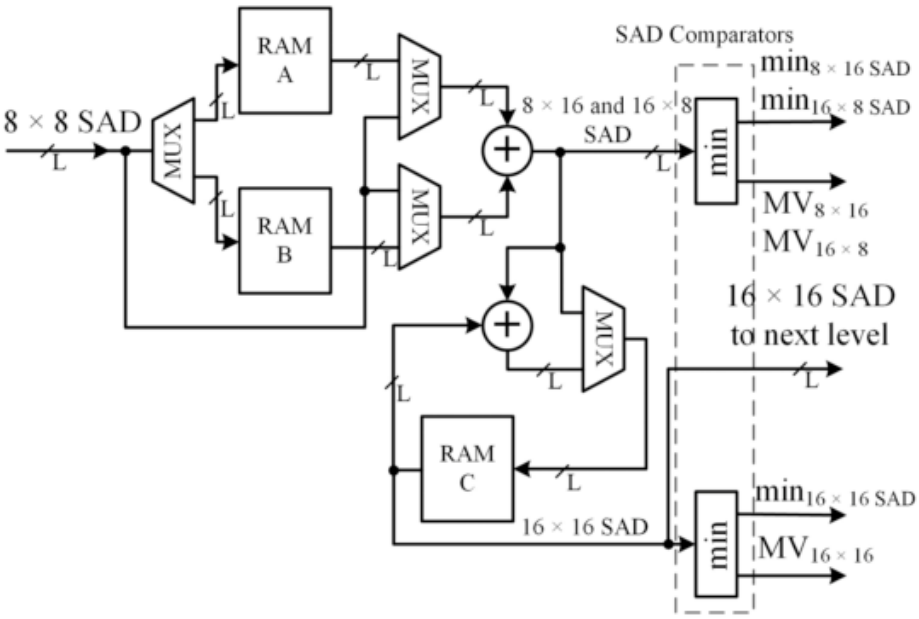


Figure 5: Image of the original summation block used in [3].

Modified Design

This original architecture is very large, but there are a number of changes that can be made to make the design significantly smaller:

1. Instead of using a 64x64 search window (as is done in the paper), my design uses a much smaller 32x32 search window. Immediately, this halves the width of my datapath and reduces the size of every SRAM to one quarter of what it was originally.
2. Instead of implementing the entirety of HEVC/H.265, my design implements AVC/H.264. This allows us to ignore block sizes larger than 16x16 and cut two of the three SAD summation units and six of the nine SRAM cells.
3. My design reworks the SAD computation module to use only two PE arrays instead of four. This reduces total cell area by around 40%, but halves the throughput.

The modified SAD computation module removes the lower two PE arrays and uses two SRAMs to compute the full 4x4, 4x8, 8x4, and 8x8 SADs in two passes. The overall idea is quite similar to that of the SAD summation module, but the implementation is quite different due to the fact that two 4x4 SAD blocks are generated on every pass (as opposed to only one in the summation unit).

In the first pass, the computation module computes the upper two 4x4 SAD blocks and the upper 4x8 SAD block while storing the 4x4 SAD blocks into the two SRAMs. In the second pass, the computation module computes the remaining 8x4, 4x8, and 8x8 SAD blocks from those being computed currently and those in memory. The complete, reworked computer module is below:

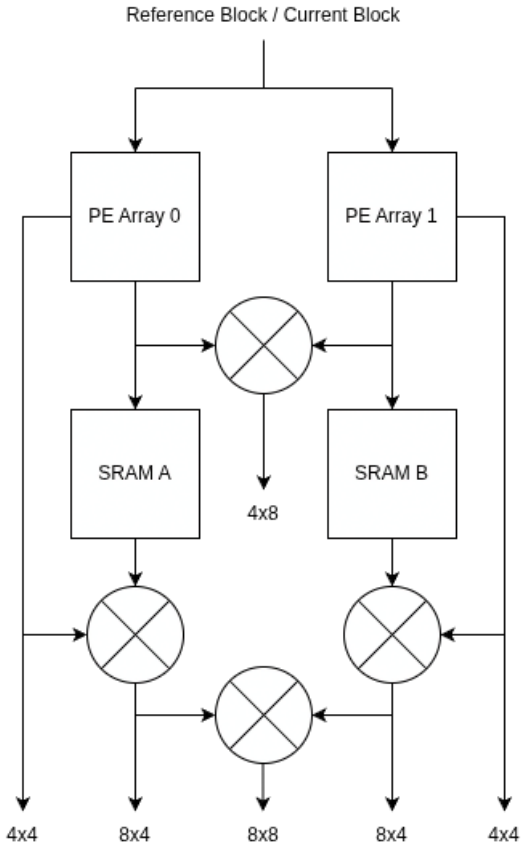


Figure 6: New compute module. Reworked for the modified architecture.

Verification

There is no rounding or truncation anywhere in the motion estimation datapath. Additionally, because we are using the full-search algorithm, the motion vector produced by the design is always guaranteed to be the minimum motion vector over the search window. Our verification doesn't deal with evaluating the encoding efficiency; this is a property of the full search algorithm itself and not of our implementation. The objective of our verification is to ensure the logical correctness of our implementation. The design is verified against a golden model which outputs all of the minimum SAD values in the current and reference frames.

The design was verified with a series of module-level unit tests and one complete functional test. All tests are implemented in the SVUnit testing framework. Current and reference frames are entirely random and therefore are not representative of real video.

1. `fs_pe_unit_test.sv` - Tests the PE module by streaming random search window pixel data.
2. `fs_pe_array_unit_test.sv` - Tests the PE array by streaming random search window pixel data.
3. `fs_computer_unit_test.sv` - Tests the computation module by streaming random pixel data.
4. `fs_comparator_unit_test.sv` - Tests the comparator module by streaming random SAD data.
5. `fs_combiner_unit_test.sv` - Tests the combiner module by streaming random SAD data.
6. `fs_control_unit_test.sv` - Tests the complete system on random CB and RB frames.

All SVUnit tests are self-checking, and all tests pass.

```
INFO:  [35936][fs_combiner_ut]: PASSED (2 of 2 tests passing)
INFO:  [35936][fs_comparator_ut]: PASSED (2 of 2 tests passing)
INFO:  [35936][fs_computer_ut]: PASSED (5 of 5 tests passing)
INFO:  [35936][fs_control_ut]: PASSED (1 of 1 tests passing)
INFO:  [35936][fs_pe_array_ut]: PASSED (5 of 5 tests passing)
INFO:  [35936][fs_pe_ut]: PASSED (3 of 3 tests passing)

INFO:  [35936][______src_tb_module_ts]: PASSED (6 of 6 testcases passing)

INFO:  [35936][testrunner]: PASSED (1 of 1 suites passing) [SVUnit Unreleased]
```

Key screenshots from the test waveforms are below.

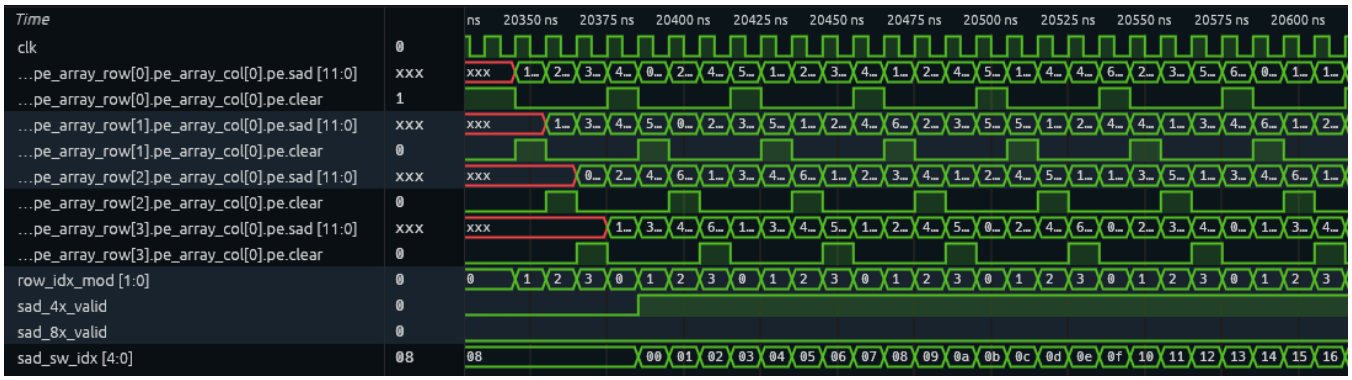


Figure 7: Output of the `fs_computer` module. Data is streamed into the PE array; the accumulators build the 4x4 SADs and are cleared every four cycles. The output is a continuous stream of SADs



Figure 8: Output of the top-level functional test with random frame data. Optimal motion vectors are collected in the comparators, and "Valid" is asserted when all possible motion vectors are tested.

Results

Theoretical Performance

The design is capable of handling a constant stream of rows from the reference frame. Rows can be streamed in from the reference frame without breaks or stalls. No flush or stall cycles are required when switching to new candidate blocks or beginning new frames. The throughput of the design is therefore solely limited by the nubmer of reference frame rows that must be sent by memory and the rate at which they are sent.

To compute a 4x4 SAD over a 32x32 search window, 35 rows of the reference frame must be streamed into the `fs_compute` module over 35 cycles. Because the FS compute module produces two 4x4 SADs at once, it takes 8 passes (280 cycles) to compute all of the 4x4 SADs for a 16x16 macroblock. Below are the required clock frequencies that would be needed to support various common input resolutions and frame rates.

Resolution	Frame Rate	Macroblock Count	Clock Frequency	Memory Bandwidth
1920x1080	30 Hz	8100	68.04 MHz	0.332 GB/s
1920x1080	60 Hz	8100	136.08 MHz	0.663 GB/s
3840x2160	30 Hz	32400	272.16 MHz	1.327 GB/s
3840x2160	60 Hz	32400	544.32 MHz	2.654 GB/s

These results are consistent with the reference design (which requires an operating frequency of 282 MHz to process 2160p video in real time). Halving the search window reduced the bandwidth and the number of cycles per pass by a factor of 2 (doubled throughput). Halving the number of PE arrays halved our throughput. The overall effect is that the number of cycles required to process a macroblock does not change substantially between the two designs.

The main drawback to the modified design is a drop in encoding efficiency. Removing support for block sizes larger than 16x16 significantly reduced encoding efficiency for static regions. The smaller search window also introduces the possibility of missing fast motion. This is especially a problem at high resolution and low framerate. It is quite likely that encoding efficiency will drop for 2160p, 30Hz.

Synthesis Overview

I synthesized the design using both with the original four-PE-array computer design from [3] and with my proposed two-PE-array computer design. As I do not have access to a memory compiler, SRAMs were modeled as black boxes with a bit density of $0.130\mu\text{m}^2$ per bit.

The synthesized area, gate count, and clock frequency for the three different designs are below. Note that gate count is provided not as leaf cell count but in number of NAND2 gate equivalents. In TSMC16, the ND2D1 cell that was used has a cell area of $0.207\mu\text{m}^2$.

Metric	Unit	Original	Modified	Reference
Process		TSMC16	TSMC16	TSMC32
Gate Count	k	509	308	2878
Clock Frequency	MHz	800	800	282
Total Cell Area	μm^2	105458	63828	

As demonstrated in the table above, removing support for blocks larger than 16x16 and reducing the search window to 32x32 cut the gate count by a factor of 5.65x. Replacing the original four-PE-array computer with the modified two-PE-array computer further reduced the gate count by 39.5%. In total, my proposed design is 9.3 times smaller than the original.

These area savings come at a cost. As discussed in [4], HEVC/H.265 achieves around a 50% bit-rate reduction over AVC/H.264. Additionally, the reduced search window size significantly reduces encoding quality for fast-moving objects at high resolutions.

Synthesis Breakdown

The complete area report is below.

```
Number of ports:                        35187
Number of nets:                        178235
Number of cells:                       131215
Number of combinational cells:         122464
Number of sequential cells:              8464
Number of macros/black boxes:               5
Number of buf/inv:                      24084
Number of references:                       1

Combinational area:              45881.874401
Buf/Inv area:                     3764.620967
Noncombinational area:            7896.061544
Macro/Black Box area:            10050.000000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                 63827.935944
Total area:                 undefined
```

The hierarchical area breakdown for the synthesized design is below.

```
****************************************
Design: fs_control
****************************************
Reference          Library       Unit Area   Count    Total Area   Attributes
--------------------------------------------------------------------------------
AN2D1BWP20P90      N16ADFP_StdCelltt0p8v25c_ccs     0.259200       1     0.259200
AO21D1BWP20P90     N16ADFP_StdCelltt0p8v25c_ccs     0.311040       3     0.933120
...
fs_combiner                   10425.557834       1  10425.557834  b, h, n
fs_comparator_SAD_RESULT_W12_0   453.237125       1   453.237125  h, n
fs_comparator_SAD_RESULT_W12_1   453.237125       1   453.237125  h, n
fs_comparator_SAD_RESULT_W13_0   516.844809       1   516.844809  h, n
fs_comparator_SAD_RESULT_W13_1   516.844809       1   516.844809  h, n
fs_comparator_SAD_RESULT_W13_2   516.844809       1   516.844809  h, n
fs_comparator_SAD_RESULT_W14    558.524170       1    558.524170  h, n
fs_comparator_SAD_RESULT_W15    567.025927       1    567.025927  h, n
fs_comparator_SAD_RESULT_W16    662.100492       1    662.100492  h, n
fs_computer                   48599.713077       1  48599.713077  b, h, n
--------------------------------------------------------------------------------
Total 39 references                                  63827.935944
```

Note that the size of the design is dominated by the size of the computer and combiner modules. Within these modules, the PE arrays have areas of around $20500 \mu\text{m}^2$ and SRAMs have areas between $1800 \mu\text{m}^2$ and $2250 \mu\text{m}^2$

The design meets timing at 800 MHz. The worst-case negative slack is zero. The tool identifies two primary critical paths: the high-fanout synchronous clear signal broadcast to the PE arrays and the second stage of the pipelined min reduction tree that implements the comparator module. The synthesizer aggressively optimized these paths for area, so the slack is zero for both.

```
  Startpoint: control/rb_row_idx_q_regx4x
              (rising edge-triggered flip-flop clocked by CLK)
  Endpoint: control/computer/pe_array_l/pe_array_rowx0x_pe_array_colx9x_pe/
acc_q_regx11x
            (rising edge-triggered flip-flop clocked by CLK)
  Path Group: reg2reg
  Path Type: max
...
  Startpoint: control/comparator_16x16/sad_l2_q_regx1xx1x
              (rising edge-triggered flip-flop clocked by CLK)
  Endpoint: control/comparator_16x16/result_col_q_regx3x
            (rising edge-triggered flip-flop clocked by CLK)
  Path Group: reg2reg
  Path Type: max
```

The power report states that the total power of the design is 45.830 mW.

```
------------------------------------------------------------------------------
                                       Switch   Int      Leak     Total
Hierarchy                              Power    Power    Power    Power    %
------------------------------------------------------------------------------
me_top                                   12.533   33.199 9.85e+04   45.830 100.0
```

The design report is below as well.

```
Library(s) Used:

    N16ADFP_StdCelltt0p8v25c_ccs (File: /soft/ece_soft_2/TSMCHOME/
Executable_Package/Collaterals/IP/stdcell/N16ADFP_StdCell/CCS/
N16ADFP_StdCelltt0p8v25c_ccs.db)

Local Link Library:

    {N16ADFP_StdCelltt0p8v25c_ccs.db}

Flip-Flop Types:

    No flip-flop types specified.

Latch Types:

    No latch types specified.

Operating Conditions:


    Operating Condition Name : tt0p8v25c
    Library : N16ADFP_StdCelltt0p8v25c_ccs
    Process :   1.00
    Temperature :  25.00
    Voltage :   0.80
    Interconnect Model : balanced_tree

Wire Loading Model:

    Selected automatically from the total cell area.

Name           :   ZeroWireload
Location       :   N16ADFP_StdCelltt0p8v25c_ccs
Resistance     :   1e-05
Capacitance    :   1
Area           :   0
Slope          :   0
Fanout   Length   Points Average Cap Std Deviation
```

```
------------------------------------------------------------
     1     0.00
     2     0.00
     3     0.00
     4     0.00
     5     0.00
     6     0.00
     7     0.00
     8     0.00
     9     0.00
    10     0.00
    11     0.00
    12     0.00
    13     0.00
    14     0.00
    15     0.00
    16     0.00
    17     0.00
    18     0.00
    19     0.00
    20     0.00

Wire Loading Model Mode: segmented.

Timing Ranges:

    No timing ranges specified.

Pin Input Delays:

    None specified.

Pin Output Delays:

    None specified.

Disabled Timing Arcs:

    No arcs disabled.

Required Licenses:

    None Required

Design Parameters:

    None specified.
1
```

References

[1] S. K. Chatterjee and I. Chakrabarti, "Low power VLSI architectures for one bit transformation based fast motion estimation," *IEEE Transactions on Consumer Electronics*, vol. 56, no. 4, pp. 2652–2660, 2010, doi: 10.1109/TCE.2010.5681153.

[2] O. Ndili and T. Ogunfunmi, "Algorithm and Architecture Co-Design of Hardware-Oriented, Modified Diamond Search for Fast Motion Estimation in H.264/AVC," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 9, pp. 1214–1227, 2011, doi: 10.1109/TCSVT.2011.2133990.

[3] N. Cheeckottu Vayalil and Y. Kong, "VLSI Architecture of Full-Search Variable-Block-Size Motion Estimation for HEVC Video Encoding," *IET Circuits, Devices & Systems*, vol. 11, no. 6, pp. 543–548, 2017, doi: https://doi.org/10.1049/iet-cds.2016.0267.

[4] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the High Efficiency Video Coding (HEVC) Standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012, doi: 10.1109/TCSVT.2012.2221191.